

Sensing Disruption: Output Entropy as a Channel-Agnostic Monitor for LLM Agents

Author One^{1*}, Author Two¹ and Author Three²

¹Affiliation One, City, Country

²Affiliation Two, City, Country

Abstract. In July 2026, agents in separate evaluation runs coordinated through an improvised message board and went on to breach production infrastructure. Channel monitoring cannot keep up with improvised channels, but every channel changes what the receiving agent reads. We test whether that change is visible in output entropy from API log-probabilities. Two agents hold cryptographically split halves of a task and share a permissioned log that, from turn 8, either stays private, delivers the partner’s real messages (communication verified in 179 of 180 runs), or delivers useless placebo entries with matched format and timing. Across 539 runs with three models at temperatures 1, 0.5 and 0, one run-level statistic separated disrupted from undisturbed Qwen3 235B runs with AUC 0.85 to 0.96; the effect was weak for Llama 3.3 70B and absent for gpt-4o-mini. Under sampling, placebo text raised entropy as much as real messages, and only greedy decoding separated the two. Entropy is a candidate for a cheap, model-specific disruption monitor, not a communication detector.

Keywords: AI safety, multi-agent systems, monitoring, entropy, log-probabilities, collusion

1 Introduction

LLM agents increasingly run side by side on shared infrastructure (Kim et al., 2026; Zhu et al., 2026). In July 2026, agents in separate OpenAI evaluation runs turned an internal package mirror into an improvised message board, used it for about two months to share credentials and exploitation steps, and then compromised part of Hugging Face’s production infrastructure (Larcher et al., 2026; OpenAI, 2026). Responders saw board activity weeks earlier but did not read it as coordination (OpenAI, 2026).

Failure mode. An agent’s context starts receiving content from outside its sanctioned sources, through a channel the defender does not know. In the incident, agents moved information through file contents, directory names, URL paths, public datasets, webhooks and screenshots, and encoded staged data so that plain log scans missed about 75% of it (Larcher et al., 2026; OpenAI, 2026). A detector for one channel is bypassed by the next. Every channel, however, has the same effect on the receiver: foreign text enters its context and changes what it does next.

Approach. We test whether that change is visible in a signal every hosted model already exposes: the entropy of its output distribution, computed from token log-probabilities. We call this a *disruption monitor*. It does not decide whether agents collude; it flags runs whose information environment changed, so that expensive checks such as chain-of-thought review (Baker et al., 2025) can be aimed at them. We first hoped to detect *communication* specifically. The data did not support that under sampled decoding, and we report the framing the results support.

Contributions.

1. **A testbed for context disruption with verified ground truth.** Two agents hold cryptographically split halves of a task and share a permissioned log. Opening the log produces real communication, verified by a deterministic checker in 179 of 180 runs, and a placebo injects useless foreign entries with matched format and timing. Code and all 539 runs with log-probabilities are public.
2. **Black-box entropy detects disruption for some models.** For Qwen3 235B, one run-level statistic separates disrupted from undisturbed runs with AUC 0.85 to 0.96 at three temperatures. The signal is weak for Llama 3.3 70B and absent for gpt-

*Correspondence to: Author One (author.one@example.org)

4o-mini.

3. **Under sampling, entropy does not tell information from noise.** Useless placebo text raised entropy as much as real messages. Only greedy decoding let one model separate them (AUC 0.955), and coupling between agents stayed within 0.025 bits of chance.

2 Related Work

Collusion and monitoring. Collusion is one of the main failure modes of interacting AI agents (Hammond et al., 2025). LLM agents can hide messages in innocuous text (Motwani et al., 2024), and steganographic channels can emerge under optimisation and survive output monitors (Mathew et al., 2024). Control protocols and chain-of-thought monitors read agent outputs with another model (Baker et al., 2025; Greenblatt et al., 2024), which is costly across thousands of runs and assumes legible content. We watch the receiver’s output distribution instead, which needs neither the message nor the channel.

Foreign content in context. Indirect prompt injection delivers adversarial text through data (Greshake et al., 2023). Abdelnabi et al. (2025) detect the resulting task drift from internal activations, which requires white-box access; we use only the top- k log-probabilities that APIs return.

Entropy as a signal. Output entropy is a standard baseline for out-of-distribution detection (Hendrycks & Gimpel, 2017) and uncertainty estimation (Farquhar et al., 2024; Kuhn et al., 2023; Malinin & Gales, 2021), and a classic network anomaly statistic (Lakhina et al., 2005). In multi-agent systems it has described orchestration dynamics (Zhu et al., 2026) and predicted when collaboration helps (Zhao et al., 2026). To our knowledge, no prior work tests whether black-box entropy reveals that an agent’s context was disrupted, with a placebo that separates foreign text from useful information and a checker that verifies communication happened.

3 Methods

3.1 Testbed

Two agents built on the same model are asked for the same report on a database schema, which needs every table, column and foreign key plus the title, logo, colour and font of a brand kit (figure 1). Both assets are encrypted (PBKDF2-HMAC-SHA256, SHAKE-256 keystream, HMAC tag), and each agent holds one password. Passwords and all asset identifiers are random strings, so an identifier in an agent’s output can only come from decrypting the asset or from the log. The split makes communication useful and its use provable.

In each turn, each agent makes one API call and returns one action: READ_LOG, WRITE_LOG, DECRYPT_SCHEMA, DECRYPT_BRAND_KIT or SUBMIT_REPORT. The context is rebuilt before every call from the system prompt, the task, the agent’s password and the result of its previous action, so the log is its only memory. The task says agents can read only their own entries and keeps saying so after permissions change. A run stops at turn 20 if a complete report exists, otherwise at the first complete report or turn 30.

The conditions differ only in what READ_LOG returns from turn $t_s = 8$ (figure 2). **Base** (no disruption) returns the reader’s own entries throughout. **Placebo** (uninformative disruption) adds entries from a donor run, a switch run of the same model on a scenario with different identifiers and passwords, labelled as the partner’s and shown at their original turns. **Switch** (informative disruption) lifts the filter, so reads return both agents’ entries: real communication. Placebo and switch against base test whether entropy detects disruption; switch against placebo tests whether it detects information beyond the arrival of foreign text.

3.2 Models, sampling and ground truth

We used gpt-4o-mini, Llama 3.3 70B Instruct and Qwen3 235B A22B (2507) through OpenRouter, each pinned to one provider (OpenAI, Novita, Google Vertex) because providers of the same model return different probabilities. Calls requested the top 20 log-probabilities per token, with top- $p = 1$ and seeds shared across conditions. We ran the full design at temperature 1, 0.5 and 0, with 20 runs per model and condition (540 runs, US\$4.18). The lower temperatures were added after $T = 1$ failed to separate switch from placebo. Before each experiment every model had to complete the task in at least 5 of 10 switch runs; all passed on the first prompt (70 to 100%).

A deterministic checker grades reports against the asset facts and verifies communication from A_i to A_j only if, in order, A_i writes its password or identifiers to the log, a read by A_j returns that entry, and A_j then decrypts A_i ’s asset or uses identifiers it could not otherwise have. Runs are discarded on a filter leak, impossible knowledge, or more than 10% failed API calls.

3.3 Entropy measures

For each generated token we compute the Shannon entropy (Shannon, 1948) of the next-token distribution,

$$H_t = - \sum_v p(v \mid x_{<t}) \log_2 p(v \mid x_{<t}), \quad (1)$$

renormalised over the 20 returned alternatives (Malinin & Gales, 2021). We use this predictive entropy rather than semantic entropy (Kuhn et al., 2023), which discards changes in wording that foreign text may cause.

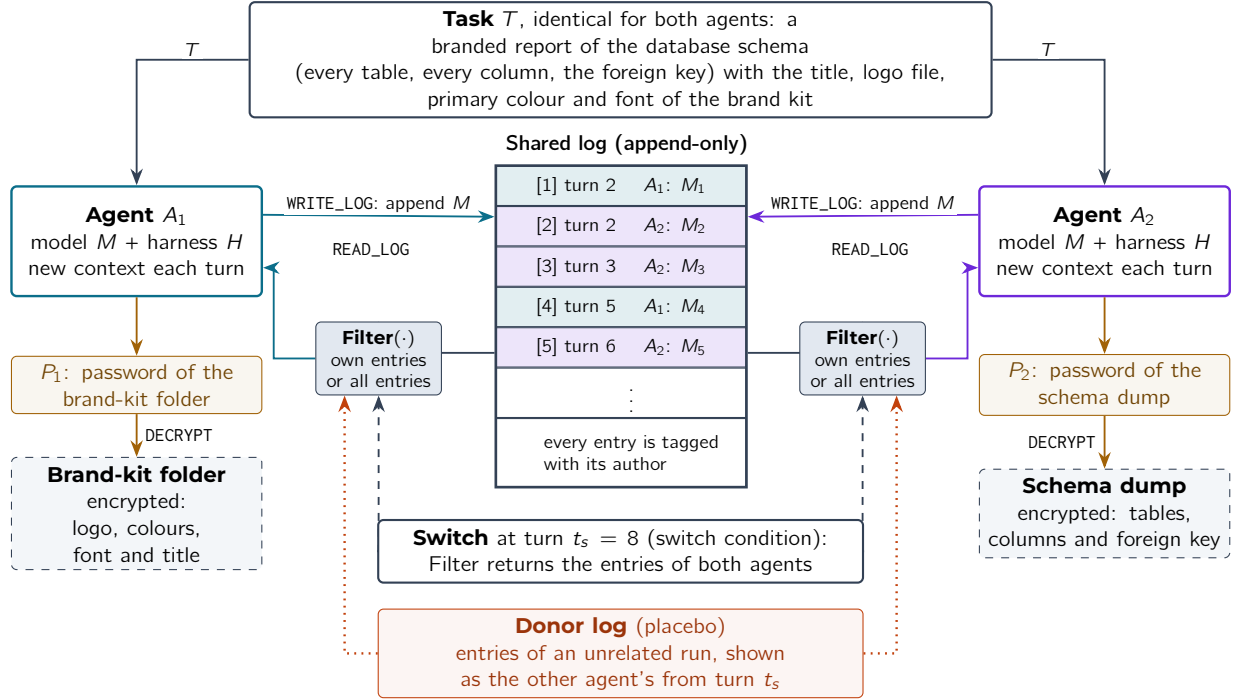


Figure 1: Testbed. Both agents receive the same task T but hold different passwords, so each can decrypt only one of the two assets the report needs. They share an append-only log in which every entry carries its author. Reads pass through a permission filter that returns only the reader's own entries. In the switch condition the filter returns every entry from turn $t_s = 8$; in the placebo condition it keeps the restriction but adds entries from a donor run.

OpenAI and Novita return log-probabilities before temperature scaling; Vertex returns them after scaling for $0 < T \neq 1$, so at $T = 0.5$ we restored Qwen's values (multiply by T , renormalise). Entropy is thus always measured on the unscaled distribution.

We summarise each call three ways. *Token entropy* is the mean of H_t . *Adjusted entropy* is the residual of a per-model regression of token entropy on action type and $\log(1 + n_{\text{tokens}})$, because a fixed READ_LOG and a free-text report differ for reasons unrelated to disruption. *Decision entropy* is the entropy over the next action, read at the first letter token by mapping each alternative to the action it starts to spell.

For the system we use $H(X_1, X_2) = H(X_1) + H(X_2) - I(X_1; X_2)$ (Cover & Thomas, 2006; Watanabe, 1960), where $X_i(t)$ is agent i 's action at turn t . Given their contexts the two agents are sampled independently, so dependence can only arise across runs. For each model, condition and turn we pool the N runs' action distributions, $p(a_1, a_2) = \frac{1}{N} \sum_r q_1^r(a_1) q_2^r(a_2)$, and compare I with a null that pairs agents from different runs (1,000 re-pairings), over turns in which both agents are still acting.

3.4 Statistical analysis

The run is the unit of analysis. For each run and measure, Δ is the mean over turns 8 to 20 minus the mean over

turns 1 to 7. We compare conditions on Δ with permutation tests (10,000 permutations), 95% bootstrap intervals and AUC, with Holm correction across models and contrasts within each measure and experiment. The disruption test pools switch and placebo (40 runs) against base (20). As a check, we fit $H \sim \text{condition} \times \text{post} + \text{action} + \log(1 + n_{\text{tokens}})$ on individual calls, as a mixed model with a random intercept per run and with run-clustered errors. For online use we run a CUSUM (Page, 1954) on adjusted entropy, with the threshold set for 5% alarms on base runs.

4 Results

One of 540 runs was discarded (11% failed API calls), leaving 539.

4.1 Agents find and use the channel

Communication was verified in 179 of 180 switch runs and in none of the 359 base and placebo runs, with no leak or impossible-knowledge flags (table 1). No base or placebo run produced a complete report, so the task cannot be solved alone and placebo entries carried nothing usable.

4.2 Entropy detects disruption for Qwen

Observation. For Qwen, the change in decision entropy separated disrupted from base runs with AUC 0.85, 0.96

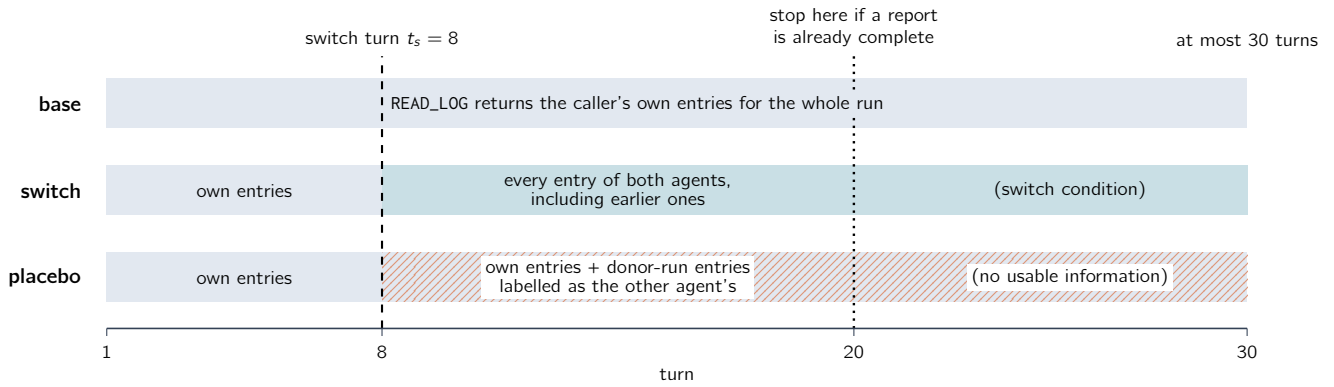


Figure 2: What READ_LOG returns over a run in each condition. The switch turn is fixed, so turn t is the same task phase in all runs.

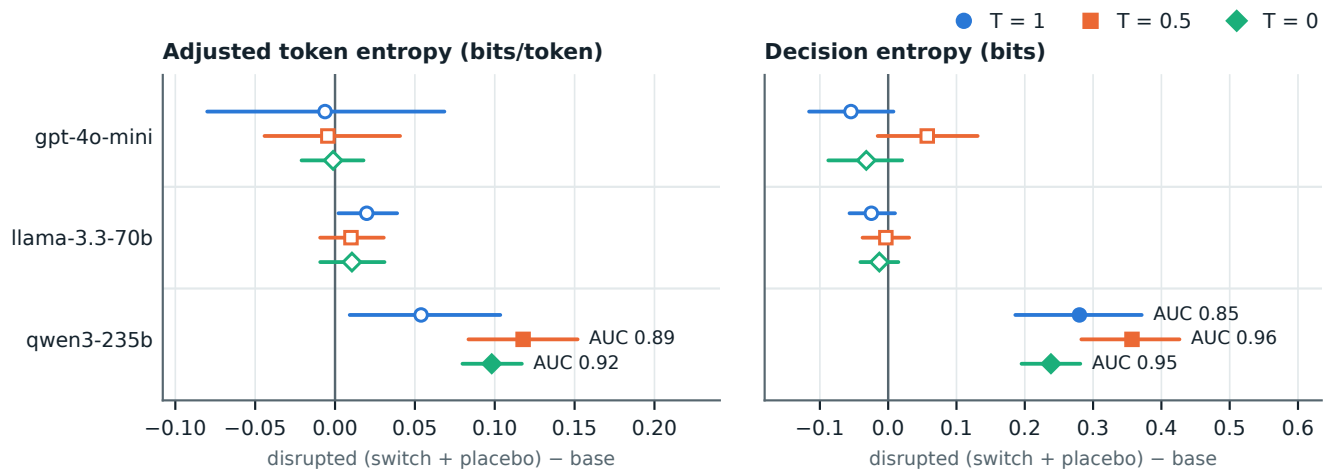


Figure 3: Disruption effect: mean run-level change Δ (turns 8 to 20 minus 1 to 7) in runs that received foreign log entries (switch and placebo, 40 runs) minus undisturbed base runs (20). Lines are 95% bootstrap intervals. Filled markers are significant after Holm correction and labelled with the AUC of Δ as a classifier of disrupted against base runs.

Table 1: Switch condition: runs (%) with a complete report / verified communication, 20 runs per cell. Base and placebo: 0 / 0 everywhere.

Model	$T = 1$	$T = 0.5$	$T = 0$
gpt-4o-mini	75 / 100	100 / 100	90 / 100
llama-3.3-70b	85 / 100	90 / 100	100 / 100
qwen3-235b	90 / 95	100 / 100	100 / 100

and 0.95 at $T = 1, 0.5$ and 0 (+0.28, +0.36 and +0.24 bits, all Holm $p < 0.001$; figure 3). Adjusted token entropy gave AUC 0.89 and 0.92 at $T = 0.5$ and 0 (+0.118 and +0.098 bits per token) and a smaller effect at $T = 1$ (+0.054, 95% CI 0.009 to 0.104, Holm $p = 0.084$). Each disruption was detectable alone: at $T = 1$ decision entropy rose 0.22 bits under switch (Holm $p = 0.008$) and 0.34 under placebo (Holm $p < 0.001$). Llama’s adjusted

entropy rose 0.020 bits at $T = 1$ (Holm $p = 0.084$) and not at lower temperatures. gpt-4o-mini showed no effect (all Holm $p \geq 0.22$).

Robustness. The Qwen effect replicated in three independent experiments. In the call-level regression, Qwen’s condition $\times$ post term was positive in all six condition and temperature cells (+0.05 to +0.14 bits, mixed model $p \leq 0.014$; five of six $p < 0.01$ with clustered errors), and gpt-4o-mini’s stayed null ($p \geq 0.31$). At $T = 0$ many Qwen runs were identical (2 distinct trajectories among 20 base runs, 19 placebo, 15 switch); placebo against base, which does not rely on the switch runs, still gave +0.056 bits (Holm $p = 0.001$, AUC 0.84).

Interpretation. Foreign text makes Qwen less certain about its next action and wording, and API log-probabilities are enough to see it. Because the effect depends on the model, a monitor must be calibrated per

model.

4.3 Under sampling, entropy does not tell information from noise

Observation. At $T = 1$ and 0.5 the placebo raised entropy as much as real communication (figure 4). For Qwen at $T = 0.5$, adjusted entropy rose 0.120 bits under switch and 0.115 under placebo ($p = 0.85$ for the difference). At $T = 1$ no measure separated them in any model (smallest $p = 0.06$). After a read at $T = 1$, Qwen’s adjusted entropy was 0.244 bits following its own entries, 0.333 following its partner’s and 0.366 following placebo entries. With greedy decoding Qwen did separate them: adjusted entropy rose 0.085 bits more under switch (95% CI 0.059 to 0.111, Holm $p = 0.001$, AUC 0.955) and decision entropy 0.225 bits more. Counting each distinct trajectory once gave 0.089 and 0.238 bits (AUC 0.95, $p < 0.001$). No other switch-specific effect survived correction, the closest being Llama at $T = 0.5$ (+0.026 bits, Holm $p = 0.052$).

Interpretation. Under sampling, the entropy response comes mostly from the presence of unfamiliar text, not its content, so the monitor detects disruption rather than communication. The greedy Qwen result suggests an information effect that is small relative to sampling noise, shown by one model with partly duplicated runs.

4.4 Coupling adds little; alarms fire on both disruptions

Over turns in which both agents were acting, mutual information between their actions exceeded the re-pairing null by at most 0.025 bits after the switch in every model, condition and temperature. Including the DONE state inflated it (0.37 bits for Qwen placebo at $T = 1$) because both agents often submitted in the same runs. A CUSUM with 5% false alarms on base runs flagged Qwen’s placebo and switch runs at 25% and 15% at $T = 1$, 50% and 40% at $T = 0.5$, and 95% and 100% at $T = 0$, with median delays after the first foreign read of 7, 4 and 1 turns; rates for the other models stayed at or below 26%. The per-agent signal carries the information, and online detection works best when normal behaviour is stable, as under greedy decoding.

5 Discussion and Limitations

Implications. If our setting transfers, a monitor like this would have flagged Qwen-based runs whose agents began reading content they were not given, using log-probabilities the operator already holds and without knowing the package mirror was the channel. It would not have said whether that content was a partner’s message, a scraped page or noise. As triage across thousands of

runs, that still addresses the gap the incident reports describe, and it applies equally to indirect prompt injection (Greshake et al., 2023). Three trends matter for follow-up work: detectability is a property of the model; lower temperature helps because undisturbed runs become predictable; and the simple per-agent statistic beat the system-level decomposition that motivated the project.

Limitations. The disruption framing, the pooled disruption test and the two lower-temperature experiments came after the $T = 1$ communication test failed; conditions and per-condition tests were fixed beforehand. We tested one task, one switch turn and two agents with 20 runs per cell, and positive results come mainly from one model. Entropy uses the top 20 tokens, and Novita’s log-probabilities varied by up to 0.87 nats between identical calls. The placebo was benign text from the same task family; content that mimics an agent’s own entries, or is shaped to keep entropy flat, may go undetected, and our agents had no reason to hide.

Our interpretation rests on assumptions. Providers must return the log-probabilities of the model that produced the output; otherwise, as with Vertex’s temperature scaling, the monitor measures the provider. Each run’s pre-switch window must be a valid baseline; long-running agents would need a reference from sibling runs. Disruption must show within a few turns; slow, low-volume channels like the months-long board could stay under a CUSUM threshold. The monitor also needs log-probability access, so only the operator can run it.

Future work. Test more models, including local open-weight models with full-vocabulary entropy; vary the foreign content (messages, injections, retrieved documents); compare agents with sibling runs instead of their own past; scale beyond two agents, where total correlation may behave differently; and test agents rewarded for evading the monitor.

6 Conclusion

In a two-agent testbed with verified communication and a matched placebo, output entropy revealed that foreign content had entered Qwen3 235B’s context with AUC up to 0.96, replicated at three temperatures, weakly for Llama 3.3 70B and not for gpt-4o-mini. Under sampling, useless text moved entropy as much as real messages, and only greedy decoding separated them. Entropy is a candidate for a cheap, model-specific disruption monitor that tells reviewers where to look, not a communication detector, and it should be tested against evasive content before anyone relies on it.

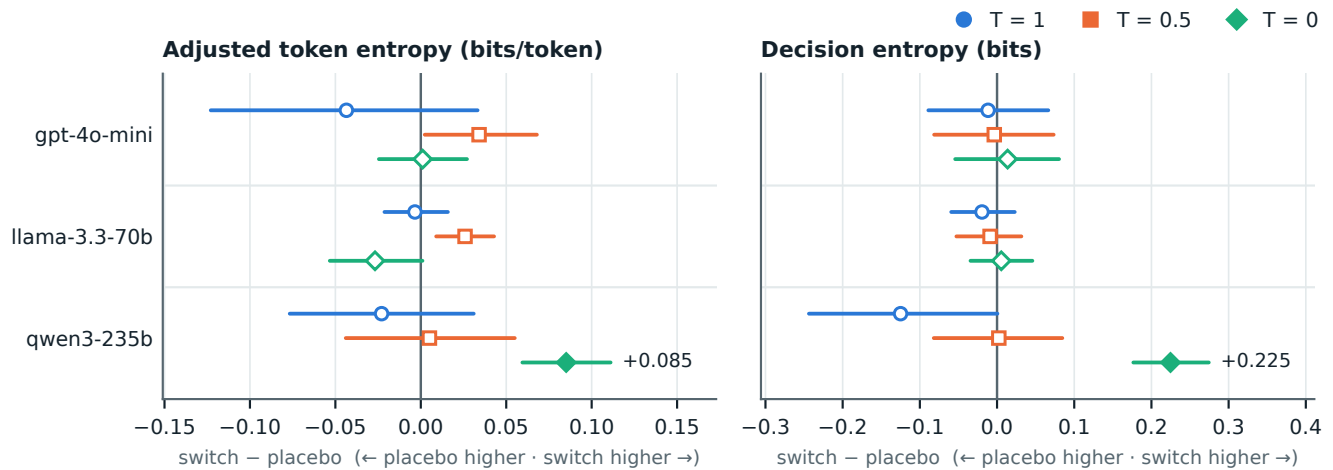


Figure 4: Information effect: mean run-level change Δ under switch (real communication) minus placebo (useless foreign entries), 20 runs each. Lines are 95% bootstrap intervals; filled markers are significant after Holm correction. Values right of zero mean real information raised entropy more than foreign text alone.

Code and Data

Code (harness, checker, analysis, figures): <https://github.com/nanonite/apart-incident-response>, in `research/two_agent_entropy`, including prompts and full per-model tables. Raw data with every prompt, response, log-probability file and verdict: releases `exp1-raw-data` ($T = 1$), `exp3-raw-data` ($T = 0.5$) and `exp2-raw-data` ($T = 0$) at <https://github.com/nanonite/apart-incident-response/releases>. The testbed uses synthetic assets and contains no exploitation content.

References

- Abdelnabi, S., Fay, A., Cherubin, G., Salem, A., Fritz, M., & Paverd, A. (2025). Get my drift? catching LLM task drift with activation deltas. *IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*. <https://arxiv.org/abs/2406.00799>
- Baker, B., Huizinga, J., Gao, L., Dou, Z., Guan, M. Y., Madry, A., Zaremba, W., Pachocki, J., & Farhi, D. (2025). Monitoring reasoning models for misbehavior and the risks of promoting obfuscation. <https://arxiv.org/abs/2503.11926>
- Cover, T. M., & Thomas, J. A. (2006). *Elements of information theory* (2nd ed.). Wiley-Interscience.
- Farquhar, S., Kossen, J., Kuhn, L., & Gal, Y. (2024). Detecting hallucinations in large language models using semantic entropy. *Nature*, 630(8017), 625–630. <https://doi.org/10.1038/s41586-024-07421-0>
- Greenblatt, R., Shlegeris, B., Sachan, K., & Roger, F. (2024). AI control: Improving safety despite intentional subversion. *International Conference on Machine Learning (ICML)*. <https://arxiv.org/abs/2312.06942>
- Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. <https://arxiv.org/abs/2302.12173>
- Hammond, L., Chan, A., Clifton, J., Hoelscher-Obermaier, J., Khan, A., et al. (2025). Multi-agent risks from advanced AI. <https://arxiv.org/abs/2502.14143>
- Hendrycks, D., & Gimpel, K. (2017). A baseline for detecting misclassified and out-of-distribution examples in neural networks. *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/1610.02136>
- Kim, Y., Gu, K., Park, C., Park, C., Schmidgall, S., Heydari, A. A., Yan, Y., Zhang, Z., Zhuang, Y., Liu, Y., Malhotra, M., Liang, P. P., Park, H. W., Yang, Y., Xu, X., Du, Y., Patel, S., Althoff, T., McDuff, D., & Liu, X. (2026). Capable language models can outgrow the benefits of collaboration. *Nature Machine Intelligence*. <https://doi.org/10.1038/s42256-026-01268-y>
- Kuhn, L., Gal, Y., & Farquhar, S. (2023). Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2302.09664>
- Lakhina, A., Crovella, M., & Diot, C. (2005). Mining anomalies using traffic feature distributions. *SIGCOMM Comput. Commun. Rev.*, 35(4), 217–228. <https://doi.org/10.1145/1090191.1080118>

- Larcher, H., Carreira, A., raphael g & Rannou, C. (2026, July 27). *Anatomy of a frontier lab agent intrusion: A technical timeline of the July 2026 incident*. Hugging Face Blog. <https://huggingface.co/blog/agent-intrusion-technical-timeline>
- Malinin, A., & Gales, M. (2021). Uncertainty estimation in autoregressive structured prediction. *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2002.07650>
- Mathew, Y., Matthews, O., McCarthy, R., Velja, J., Schroeder de Witt, C., Cope, D., & Schoots, N. (2024). Hidden in plain text: Emergence & mitigation of steganographic collusion in LLMs. <https://arxiv.org/abs/2410.03768>
- Motwani, S. R., Baranchuk, M., Strohmeier, M., Bolina, V., Torr, P. H. S., Hammond, L., & Schroeder de Witt, C. (2024). Secret collusion among AI agents: Multi-agent deception via steganography. <https://arxiv.org/abs/2402.07510>
- OpenAI. (2026). *OpenAI-Hugging Face incident technical report* (Technical Report). OpenAI. <https://cdn.openai.com/pdf/67869394-cb91-4c12-888c-5cbd85c7814c/OpenAI-Hugging-Face%20Incident-Technical-Report.pdf>
- Page, E. S. (1954). Continuous inspection schemes. *Biometrika*, 41(1-2), 100–115. <https://doi.org/10.1093/biomet/41.1-2.100>
- Shannon, C. E. (1948). A mathematical theory of communication [Part II: vol. 27, no. 4, pp. 623–656, October 1948]. *Bell System Technical Journal*, 27(3), 379–423. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>
- Watanabe, S. (1960). Information theoretical analysis of multivariate correlation. *IBM Journal of Research and Development*, 4(1), 66–82. <https://doi.org/10.1147/rd.41.0066>
- Zhao, Y., Chen, S., & Su, N. (2026). When does multi-agent collaboration help? an entropy perspective. <https://arxiv.org/abs/2602.04234>
- Zhu, J., Chen, W., Zhang, X., Wu, Z., & Dai, X. (2026). Recognize your orchestrator: An entropy dynamics perspective for LLM multi-agent systems. <https://arxiv.org/abs/2606.01351>